

Svara: Offline Indian Sign Language Recognition Using a Fine-Tuned INCLUDE Transformer

Anant Singhal

Department of Computer Science and Engineering (Artificial Intelligence)
KIET Group of Institutions, Ghaziabad, India
anant.2428cseai1061@kiet.edu

Aditya Singh

Department of Computer Science and Engineering (Artificial Intelligence)
KIET Group of Institutions, Ghaziabad, India
aditya.2428cseai1053@kiet.edu

Adyanshi Singh

Department of Computer Science and Engineering (Artificial Intelligence)
KIET Group of Institutions, Ghaziabad, India
adyanshi.2428cseai97@kiet.edu

Anshika Gautam

Department of Computer Science and Engineering (Artificial Intelligence)
KIET Group of Institutions, Ghaziabad, India
anshika.2428cseai410@kiet.edu

Abstract—Svara is an offline Android communication assistant that uses Indian Sign Language (ISL) recognition to convert isolated signs into useful communication labels. This pilot study investigates whether a pretrained AI4Bharat INCLUDE Transformer can be adapted to a small, communication-focused ISL pilot. Ten signs were selected from the public INCLUDE dataset, giving 167 usable videos divided into 120 training, 13 validation, and 34 test samples. Videos were converted into sequences of pose and hand landmarks with 169 temporal positions and 134 coordinate features per position. We compare a Transformer trained from scratch, a pretrained model with a frozen backbone, and full fine-tuning. Across three random seeds, the frozen pretrained model gives the strongest mean result, achieving $95.10\% \pm 1.70\%$ accuracy and $94.30\% \pm 2.25\%$ macro-F1. The selected FP32 model was exported to ONNX, matched PyTorch predictions, and was integrated into the Android application. Dynamic INT8 quantization reduced model size by 74.54% but reduced pilot accuracy to 85.29%, so FP32 is retained for deployment. These results are preliminary and are reported as a pilot rather than signer-independent or production-level performance.

Keywords—Indian Sign Language, INCLUDE Transformer, transfer learning, landmark sequence, ONNX Runtime, Android, offline recognition

I. INTRODUCTION

Communication barriers remain a practical concern for people who use Indian Sign Language (ISL), especially when the communication partner does not know ISL. Svara addresses this problem with a local-first mobile pipeline: the camera observes a short isolated sign, MediaPipe extracts pose and hand landmarks, and an ONNX model predicts a useful ISL label without sending video to a server. The scope is deliberately limited to a ten-sign pilot so that the data pipeline, evaluation protocol, and deployment path can be measured before scaling to a larger vocabulary.

The central research question is: can a pretrained INCLUDE Transformer be adapted to a curated communication-focused ISL subset and retain useful accuracy for offline Android inference? The study contributes a reproducible pilot pipeline, a comparison of pretrained and scratch training, and an initial deployment analysis using ONNX Runtime. It does not claim continuous sentence translation, signer-independent generalization, or clinical reliability.

II. DATASET AND PREPROCESSING

A. Dataset Scope

The videos were obtained from the public AI4Bharat INCLUDE dataset [1]. The pilot vocabulary contains bathroom, cellphone, doctor, hello, hospital, medicine, money, patient, sick, and thankyou. The prepared manifest contains 167 videos. The video-level split contains 120 training videos, 13 validation videos, and 34 test videos. The split is fixed before evaluation and is reused for the model comparisons. Because signer identities were not independently verified for this pilot, the test result is not described as an unseen-signer result.

B. Landmark Representation

Each video is represented by pose and hand landmarks rather than raw RGB pixels. The extracted frame contains the first 25 pose points, 21 left-hand points, and 21 right-hand points. Each point contributes an x and y coordinate, giving 134 values per frame. Temporal interpolation and padding produce a fixed sequence of 169 frames. The resulting model input is therefore a tensor with shape [169, 134]. This representation reduces storage and keeps the inference pipeline suitable for an offline phone.

C. Quality Checks

All 167 extracted videos passed the pilot quality checks: no missing files, unreadable videos, or zero-size files were recorded. Video durations range from 1.64 to 3.84 seconds, with a mean of 2.52 seconds. The class counts are not perfectly equal, so macro-averaged metrics are reported in addition to accuracy.

III. PROPOSED METHOD

A. Model Alternatives

Three training conditions are compared. The scratch Transformer is trained without pretrained INCLUDE weights. The pretrained frozen condition loads the INCLUDE Transformer representation and trains only the ten-class output head. The full fine-tuning condition initializes from the same pretrained model and updates the backbone and the new output head. Each condition is repeated with seeds 42, 123, and 2026.

B. Training and Inference Pipeline

The training pipeline loads landmark sequences, maps the ten labels to integer IDs, and optimizes a cross-entropy classifier. The best checkpoint is selected using validation loss. During inference, the sequence is normalized to the model length, the ONNX model produces ten logits, and softmax converts them into class probabilities. The Android implementation uses MediaPipe for landmark extraction and ONNX Runtime for local model execution. This design follows the general mobile deployment flow in which an ONNX model is loaded and executed on the device [2], while MediaPipe provides normalized pose and hand landmarks [3].

IV. EXPERIMENTAL SETUP

The pilot uses the fixed 120/13/34 train, validation, and test partition. The ten-class output layer is evaluated using accuracy, macro precision, macro recall, and macro-F1. Macro averaging gives every sign equal importance even when class counts differ. The comparison is repeated across three seeds to report mean and standard deviation. The final deployment candidate is exported from PyTorch to ONNX and checked by comparing its predicted class with the PyTorch prediction on the same keypoint input.

V. PRELIMINARY RESULTS

Table I shows the three-seed pilot comparison. The frozen pretrained condition provides the highest mean accuracy and macro-F1. Full fine-tuning does not improve over the frozen backbone on this small pilot, while training from scratch is less accurate and more variable. The result supports using transfer learning as the default configuration for the next stage of Svava data collection.

Model	Accuracy	Macro-P	Macro-R	Macro-F1
Full Fine-tuning	94.12 ± 0.00%	96.67 ± 0.00%	93.33 ± 0.00%	93.00 ± 0.00%
Pretrained Frozen	95.10 ± 1.70%	97.11 ± 0.77%	94.44 ± 1.92%	94.30 ± 2.25%
Scratch	91.18 ± 2.94%	93.35 ± 2.95%	90.83 ± 2.50%	90.76 ± 2.20%

TABLE I. THREE-SEED RESULTS ON THE 10-SIGN PILOT

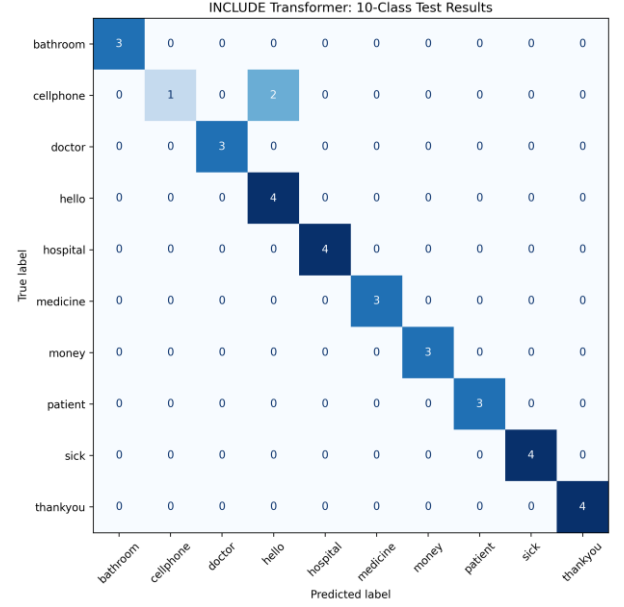


Fig. 1. Confusion matrix for the selected pilot evaluation.

VI. ONNX AND ANDROID PROGRESS

The selected frozen model was exported to ONNX with input name keypoints and shape [1, 169, 134]. The ONNX structure was valid, the PyTorch and ONNX predictions matched on verification, and the maximum output difference was 1.67×10^{-6} . The FP32 model was then placed in the Android assets and connected to the Svava recognizer. The application uses the ten pilot labels and runs inference locally through the onnxruntime-android dependency.

Dynamic INT8 quantization was evaluated as a model-size experiment. It reduced the model from 14.45 MB to 3.68 MB, but accuracy fell from 97.06% (33/34) to 85.29% (29/34), with only 88.24% prediction agreement. Therefore the FP32 model is retained as the deployment candidate.

VII. LIMITATIONS AND NEXT WORK

The pilot has a small test set of 34 videos and does not establish signer-independent generalization. It covers isolated signs rather than continuous ISL sentences, and its class vocabulary is intentionally narrow. The reported confidence is a softmax score and has not been calibrated. Phone testing is reserved for a later report because camera conditions, distance, lighting, and signer variation require a separate protocol.

The next stage will expand the vocabulary and number of signers, preserve signer-disjoint evaluation where the metadata permits it, measure latency and battery use on the target phone, and compare the Android landmark pipeline against the training preprocessing pipeline.

VIII. CONCLUSION

This pilot establishes an end-to-end path from a public ISL dataset to an offline Android recognizer. On the ten-sign subset, the pretrained frozen INCLUDE Transformer is the strongest and most stable of the three tested training conditions. The successful ONNX verification and Android integration provide a concrete deployment baseline, while the small dataset and limited signer evidence define the work required before any broader performance claim.

ACKNOWLEDGMENT

The authors thank Dr. Gaurav Srivastav for guidance and feedback during the project.

REFERENCES

- [1] A. Sridhar, R. G. Ganesan, P. Kumar, and M. Khapra, "INCLUDE: A Large Scale Dataset for Indian Sign Language Recognition," in Proc. 28th ACM Int. Conf. Multimedia, 2020, pp. 1366–1375, doi: 10.1145/3394171.3413528.
- [2] Microsoft, "ONNX Runtime mobile application development," ONNX Runtime Documentation, 2026. [Online]. Available: <https://onnxruntime.ai/docs/tutorials/mobile/>.
- [3] Google AI Edge, "Holistic landmark detection guide," MediaPipe Documentation, 2026. [Online]. Available: https://ai.google.dev/edge/mediapipe/solutions/vision/holistic_landmarker.
- [4] A. Vaswani et al., "Attention Is All You Need," in Proc. 31st Int. Conf. Neural Information Processing Systems, 2017, pp. 5998–6008.